

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Mauricio Cruz Quijada

7 de septiembre de 2026

19:16

1. Introducción

Durante esta tarea se estudia experimentalmente cómo se comportan distintos algoritmos para dos problemas específicos: El ordenamiento de arreglos en una dimensión y la multiplicación de matrices cuadradas entre ellas. Para el primer problema se utilizaron y compararon los siguientes algoritmos: MergeSort, QuickSort, PatienceSort y sort, mientras que para la multiplicación de matrices se utiliza Naive y Strassen. su objetivo es analizar los tiempos de ejecución y también de uso de memoria para diferentes tamaños y/o características en las entradas, así poder contrastar los resultados, la velocidad de obtención y el espacio necesario para ser ejecutado, todo esto dependiendo de la complejidad de cada algoritmo.

2. Entorno experimental

Los experimentos fueron realizados en un computador con procesador Intel Core i5-12500H de 12ª generación, con 16 procesadores lógicos y aproximadamente 7,6 GiB de memoria RAM disponible para el sistema. Las pruebas se ejecutaron utilizando WSL2 sobre Windows, con Ubuntu 24.04.4 LTS y kernel Linux 6.6.87.2. Los programas fueron implementados en C++17 y compilados utilizando g++ 13.3.0 con la opción de optimización -O2. Para la generación de los datasets y de las gráficas se utilizó Python 3.12.3.

Para los experimentos de ordenamiento se utilizaron arreglos de tamaños 10, 1000, 100000 y 10000000, considerando arreglos aleatorios, ascendentes y descendentes, dos dominios de valores (D1 y D7) y tres muestras por configuración. Para la multiplicación de matrices se utilizaron matrices cuadradas de tamaños 16, 64, 256 y 1024, considerando matrices densas, diagonales y dispersas, los dominios D0 y D10

y tres muestras por configuración. En ambos experimentos se registró el tiempo de ejecución y el uso máximo de memoria de cada algoritmo.

Durante las mediciones de matrices se detectaron dos duraciones negativas y una duración positiva anómala. Se sustituyó el reloj utilizado por `std::chrono::steady_clock`, se recompiló el programa y se repitieron únicamente los tres casos afectados: Naive para las matrices densas D10 de tamaño 1024, muestras a y b, y Strassen para las matrices densas D0 de tamaño 256, muestra b. Las mediciones anteriores de esos casos fueron reemplazadas por las nuevas; las demás mediciones se conservaron.

3. Resultados y Análisis

3.1. Ordenamiento de arreglos

Los resultados presentados a continuación corresponden a una selección de los casos más representativos del conjunto completo de experimentos. Las mediciones para todas las combinaciones de tamaños, tipos de entrada, dominios y muestras se encuentran registradas en los archivos CSV generados automáticamente, mientras que las figuras seleccionadas permiten destacar las principales diferencias observadas entre los algoritmos.

Podemos observar diferencias importantes en los resultados experimentales de los algoritmos de ordenamiento, especialmente a medida que aumenta el tamaño de los arreglos. Para $n = 10$ todos los algoritmos presentan tiempos muy pequeños, por lo que las diferencias entre ellos no son significativas. Sin embargo, al aumentar el tamaño de entrada estas diferencias se hacen cada vez más evidentes.

En la Figura 1 se presentan los tiempos obtenidos para arreglos aleatorios del dominio D1. En este conjunto de datos, `std::sort` [1] presentó el mejor comportamiento experimental para los tamaños mayores. Para $n = 10^7$ obtuvo un tiempo promedio de aproximadamente 0,182 s, mientras que MergeSort [2, 3] alcanzó 0,656 s y PatienceSort [6] 1,653 s. QuickSort, en cambio, superó el límite de 30 segundos en las tres muestras aleatorias del dominio D1 para este tamaño.

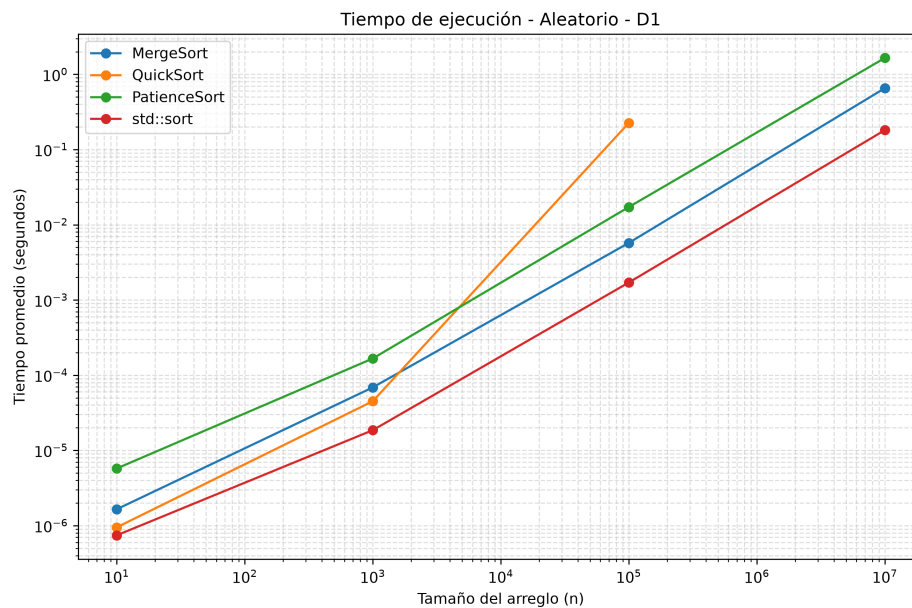


Figura 1: Tiempo de ejecución de los algoritmos de ordenamiento para arreglos aleatorios del dominio D1.

La Tabla 1 resume algunos de los tiempos promedio obtenidos para los arreglos aleatorios D1. Se observa que las diferencias entre los algoritmos se vuelven más notorias al aumentar el tamaño de entrada.

n	MergeSort	QuickSort	PatienceSort	std::sort
10^3	0.000069	0.000045	0.000167	0.000019
10^5	0.005742	0.226181	0.017213	0.001707
10^7	0.655822	TIMEOUT	1.653383	0.181568

Cuadro 1: Tiempo promedio de ejecución en segundos para arreglos aleatorios del dominio D1.

Los *timeouts* de QuickSort ocurrieron para entradas de tamaño 10^7 del dominio D1, tanto aleatorias como ascendentes y descendentes. Además, PatienceSort superó el límite de tiempo para los arreglos ascendentes de tamaño 10^7 del dominio D7. Estos resultados muestran que, además del tamaño de entrada, la distribución y el orden inicial de los datos pueden afectar de forma importante el comportamiento práctico de una implementación.

A continuación, se discuten algunos fenómenos ocurridos en el análisis de tiempo. La tendencia global es bastante clara: un aumento de n implica un aumento de su tiempo de ejecución, mas bien, este tiempo no se comporta de forma lineal. El algoritmo que sale de esta regla con creces fue el *QuickSort*, este se comportó de manera eficaz incluso con arreglos de 10^7 elementos, sin embargo, para D1 todas las ejecuciones de ese tamaño superaron el límite de tiempo. Esto lo podemos explicar mediante la forma en la que se realizaron las particiones, pues aunque se editó el código fuente para lograr que se use de pivote el elemento central, la partición de Lomuto agrupa los elementos menores e iguales al pivote en el mismo lado. Cuando existen muchos valores repetidos eso produce un desbalanceo y un aumento a las

comparaciones y llamadas recursivas, convirtiendo su tiempo en el peor caso n cuadrado. Por lo tanto, a pesar de ser un buen algoritmo, en la práctica podemos apreciar que la eficiencia no solo depende de su elección de pivote, sino también de la distribución de valores y la partición usada.

3.2. Multiplicación de matrices

En la multiplicación de matrices se observó una diferencia considerable entre el algoritmo Naive [4, 5] y la implementación de Strassen, una diferencia que se mantuvo bastante uniforme con los distintos tamaños de arreglos. La Figura 2 nos puede mostrar los distintos resultados para matrices diagonales del dominio D0. Para $n = 16$, Naive obtuvo un tiempo promedio de aproximadamente $4,27 \times 10^{-6}$ s, mientras que Strassen alcanzó $1,13 \times 10^{-3}$ s. Para $n = 64$ los promedios fueron aproximadamente $1,09 \times 10^{-4}$ s y $4,75 \times 10^{-2}$ s, respectivamente. Finalmente, para un tamaño $n = 256$, Naive obtuvo aproximadamente $6,58 \times 10^{-3}$ s, mientras que Strassen alcanzó 2,280 s.

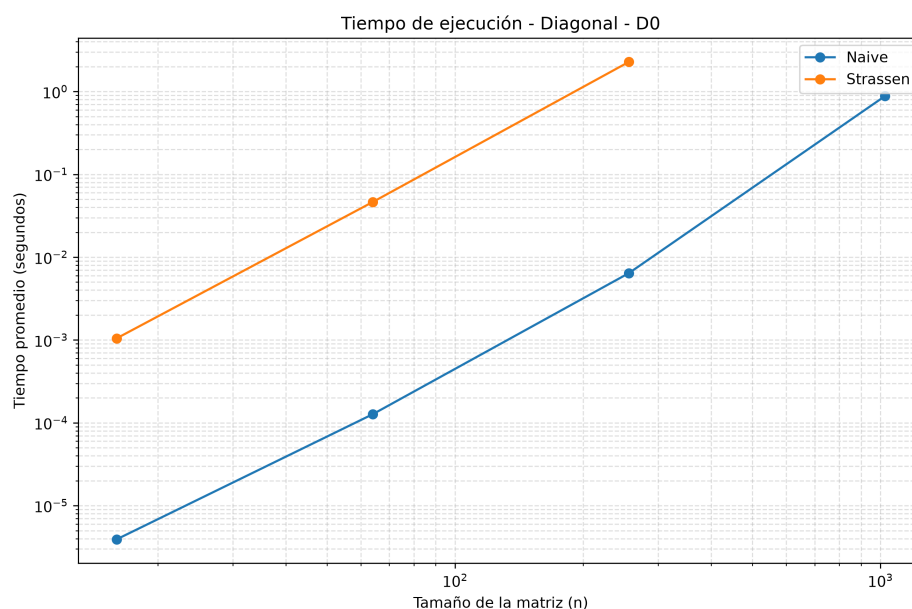


Figura 2: Tiempo de ejecución de Naive y Strassen para matrices diagonales del dominio D0.

La Tabla 2 resume estos resultados y permite observar la diferencia de tiempo entre ambos algoritmos a medida que aumenta el tamaño de las matrices.

n	Naive (s)	Strassen (s)
16	0.000003918	0.001047277
64	0.000126967	0.046354867
256	0.006411493	2.277426667
1024	0.880075667	TIMEOUT

Cuadro 2: Tiempo promedio de ejecución para matrices diagonales del dominio D0.

A pesar de que Strassen posee una mejor complejidad asintótica que el algoritmo Naive clásico, es-

ta ventaja no se observó experimentalmente en los tamaños evaluados. La implementación de Strassen incluye trabajo adicional debido a las operaciones auxiliares y a la recursión, lo que genera un costo considerable para estas entradas. Para $n = 1024$, todas las ejecuciones de Strassen, considerando los distintos tipos de matrices y dominios evaluados, superaron el límite establecido de 60 segundos. En el caso particular de las matrices diagonales D0, Naive completó las pruebas con un tiempo promedio de aproximadamente 0,855 s.

3.3. Análisis de memoria

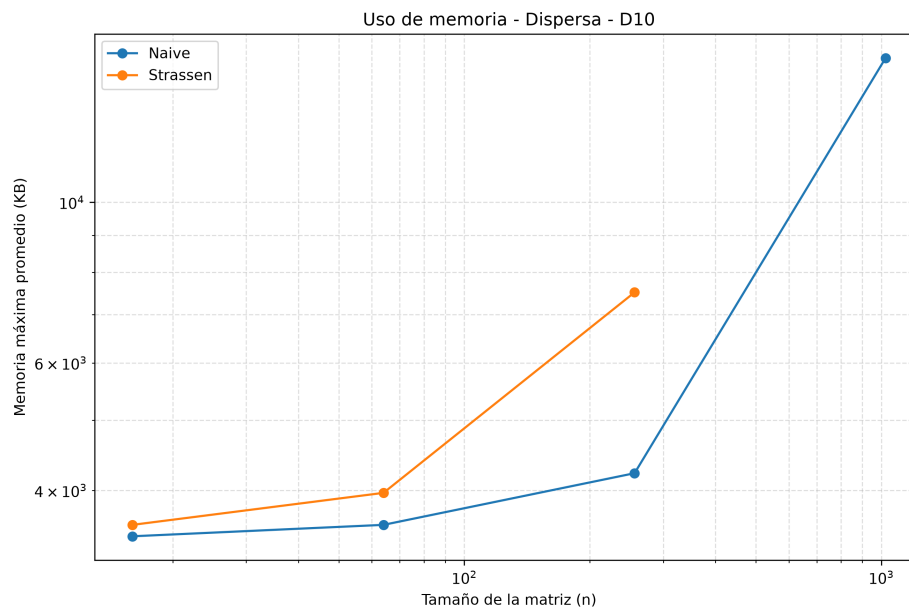


Figura 3: Uso máximo de memoria para matrices dispersas del dominio D10.

En memoria también podemos observar algunas diferencias entre los algoritmos. Para los arreglos de 10^7 elementos, MergeSort utilizó más memoria que `std::sort`, mientras que PatienceSort dependió bastante del tipo de entrada. Esto se relaciona directamente con los arreglos auxiliares de MergeSort y con la cantidad y el tamaño de las pilas que construye PatienceSort en cada una de las iteraciones. Además, las funciones de ordenamiento retornan el arreglo por valor, agregando una copia al consumo medido.

En matrices dispersas D10 de tamaño 256, Naive utilizó en promedio 4224 KB y Strassen 7509,33 KB. La diferencia es coherente con las copias y matrices auxiliares de Strassen. De todas formas, estas mediciones corresponden a la memoria máxima del proceso completo, incluyendo las entradas, y no solamente a la memoria auxiliar de cada algoritmo. Para los casos con TIMEOUT no se obtuvo una medición de memoria, por lo que no podemos compararlos directamente.

4. Conclusiones

Los resultados experimentales muestran que una mejor complejidad no necesariamente indica un menor tiempo de ejecución. En ordenamiento, `std::sort` presentó en general los menores tiempos, mientras que QuickSort y PatienceSort superaron el límite de 30 segundos en algunas entradas de tamaño 10^7 . Esto muestra que no basta con considerar la cantidad de elementos, pues también influyen su distribución y el orden inicial. En QuickSort, elegir un pivote central no evitó las particiones desbalanceadas cuando había muchos valores repetidos. Por su parte, PatienceSort mostró dificultades con las entradas ascendentes del dominio D7, donde la construcción de muchas pilas aumenta los costos de la implementación.

En multiplicación de matrices, Naive se desarrolló considerablemente más rápido que Strassen para todos los casos experimentados. Para $n = 1024$, Naive terminó las pruebas en aproximadamente 0,872 s en promedio, mientras que Strassen superó el límite de 60 segundos en todas sus ejecuciones. Aunque Strassen reduce la cantidad de multiplicaciones recursivas, las copias de matrices, las operaciones auxiliares y la recursión hasta matrices de tamaño 1 generan un costo que resultó importante en estas pruebas. Por lo tanto, estos resultados no permiten concluir que Naive siempre será mejor, sino que fue más conveniente con las implementaciones y tamaños utilizados.

En memoria también se observaron diferencias: Strassen presentó un mayor consumo que Naive en los tamaños comparables, mientras que en ordenamiento el uso de arreglos auxiliares y pilas influyó en los resultados. Sin embargo, se midió la memoria máxima del proceso completo, por lo que no todo ese consumo corresponde exclusivamente al algoritmo. Además, los casos con TIMEOUT permiten reconocer una limitación práctica, pero no conocer su tiempo final ni comparar su memoria.

En conjunto, pudimos observar que la complejidad teórica ayuda a entender el crecimiento esperado, pero no describe por sí sola todo lo que ocurre al ejecutar un programa. La elección de un algoritmo también debe considerar cómo está implementado, qué entradas recibe y cuánta memoria necesita. Como continuación, sería interesante evaluar una partición de QuickSort que agrupe los valores iguales y una versión de Strassen que utilice Naive para las matrices pequeñas, para comprobar cuánto cambian los resultados al reducir estos costos.

Referencias

- [1] cppreference.com. *std::sort*. URL: <https://en.cppreference.com/w/cpp/algorithm/sort>.
- [2] GeeksforGeeks. *C++ Program For Merge Sort*. URL: <https://www.geeksforgeeks.org/cpp/cpp-program-for-merge-sort/>.
- [3] GeeksforGeeks. *C++ Program For QuickSort*. URL: <https://www.geeksforgeeks.org/cpp/cpp-program-for-quicksort/>.
- [4] GeeksforGeeks. *C++ Program to Multiply Two Matrices*. URL: <https://www.geeksforgeeks.org/cpp/cpp-program-to-multiply-two-matrices/>.

- [5] GeeksforGeeks. *Strassen's Matrix Multiplication Algorithm*. URL: <https://www.geeksforgeeks.org/strassens-matrix-multiplication-algorithm-implementation/>.
- [6] Rosetta Code. *Sorting algorithms/Patience sort*. Implementación de la sección C++. URL: https://rosettacode.org/wiki/Sorting_algorithms/Patience_sort.